

Decision Tree Classification: Split, Pruning, dan Evaluasi

Materi Week 12 – Supervised Learning

Modul
Topik
Penulis
Mata Kuliah

Week 12: Supervised Learning
Decision Tree Classification
Danish Rafie Ekaputra
Praktikum Data Mining

Tujuan Pembelajaran

Setelah mempelajari materi ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar Decision Tree untuk classification.
2. Menjelaskan kriteria split gini dan entropy.
3. Menjelaskan penyebab overfitting pada tree.
4. Menerapkan pre-pruning dan post-pruning.
5. Mengevaluasi model dengan confusion matrix, precision, recall, F1, dan ROC-AUC.
6. Menyusun pipeline klasifikasi berbasis DecisionTreeClassifier untuk kompetisi.

Big idea. Decision Tree membagi data dengan aturan if-else yang memaksimalkan kemurnian class di setiap split. Model ini sangat interpretabel, tetapi sangat mudah overfit jika tree dibiarkan terlalu dalam tanpa pruning.

1 Kenapa Decision Tree?

Decision Tree adalah model supervised learning yang populer karena dua alasan utama: mudah dijelaskan dan fleksibel terhadap bentuk hubungan non-linear. Setiap prediksi bisa ditelusuri sebagai urutan aturan logika. Ini membantu ketika kita perlu menjawab pertanyaan praktis seperti: *“kenapa sampel ini diprediksi class 1?”*

Kondisi	Kenapa Cocok	Tree	Catatan
Butuh interpretabilitas tinggi	Aturan bisa dibaca sebagai if-else.	split dibaca	Cocok untuk materi awal supervised learning.
Hubungan non-linear	Tree tidak mengasumsikan boundary linear.	tidak	Tetap butuh kontrol kompleksitas.
Data campuran numerik+kategorikal (setelah encoding)	Tree tidak perlu scaling fitur.	perlu	Encoding tetap harus konsisten.

Table 1: Situasi umum di mana Decision Tree menjadi baseline yang kuat.

2 Struktur Decision Tree

Komponen utama:

- **Root node:** node pertama, berisi seluruh data train.
- **Internal node:** node yang melakukan split pada sebuah fitur.
- **Leaf node:** node terminal, berisi prediksi class/probabilitas.
- **Depth:** panjang jalur dari root ke leaf.

Intuisi training tree:

1. Coba banyak kandidat split.
2. Pilih split dengan impurity turun paling besar.
3. Ulangi rekursif sampai stopping criteria terpenuhi.

3 Split Criteria: Gini vs Entropy

3.1 Gini Impurity

Untuk node dengan proporsi class $p_1, p_2, \dots, p_K$:

$$Gini = 1 - \sum_{k=1}^K p_k^2 \quad (1)$$

Semakin kecil nilai Gini, node semakin "pure".

3.2 Entropy

$$Entropy = - \sum_{k=1}^K p_k \log_2(p_k) \quad (2)$$

Entropy mengukur ketidakpastian class di node. Nilai lebih kecil berarti node lebih homogen.

3.3 Information Gain

Jika split menghasilkan child kiri dan kanan:

$$IG = Entropy(parent) - \left(\frac{n_L}{n} Entropy(L) + \frac{n_R}{n} Entropy(R) \right) \quad (3)$$

Split terbaik adalah split dengan penurunan impurity terbesar.

Kriteria	Karakteristik	Praktis di scikit-learn
Gini	Komputasi sedikit lebih ringan.	<code>criterion='gini'</code> (default)
Entropy	Berbasis information gain.	<code>criterion='entropy'</code> atau <code>'log_loss'</code>

Table 2: Perbandingan ringkas kriteria split pada DecisionTreeClassifier.

4 Kenapa Tree Bisa Overfit?

Tree yang terlalu dalam dapat "menghafal" noise training data. Gejalanya:

- Train score sangat tinggi.
- Validation score stagnan atau turun.

- Banyak leaf dengan sampel sangat sedikit.

Overfitting pattern. Decision tree punya variance tinggi. Tanpa pruning, model mudah membuat aturan yang terlalu spesifik untuk training data dan gagal generalisasi pada data baru.

5 Mengontrol Kompleksitas: Pre-pruning

Parameter yang paling sering dipakai:

Parameter	Efek Utama	Trade-off
max_depth	Membatasi kedalaman tree.	Terlalu kecil bisa underfit.
min_samples_split	Node perlu sampel minimum agar boleh split.	Terlalu besar menghambat split penting.
min_samples_leaf	Leaf wajib punya sampel minimum.	Meningkatkan stabilitas probabilitas leaf.
max_features	Membatasi jumlah fitur yang dipertimbangkan tiap split.	Bisa kurangi variance, tapi juga menurunkan fit.

Table 3: Parameter pre-pruning untuk menekan overfitting.

6 Post-pruning dengan ccp_alpha

Cost complexity pruning menambahkan penalti terhadap jumlah leaf terminal.

$$R_{\alpha}(T) = R(T) + \alpha|T| \quad (4)$$

Dengan:

- $R(T)$: error tree.
- $|T|$: jumlah terminal nodes.
- α : kekuatan penalti kompleksitas.

Semakin besar ccp_alpha, tree makin sederhana.

7 Pipeline Praktis di scikit-learn

Contoh baseline yang aman:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ("num", "passthrough", numeric_cols),
    ]
)

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=6,
    min_samples_leaf=20,
    random_state=42,
```

```
)
clf = Pipeline([
    ("prep", preprocessor),
    ("model", model),
])
```

Good practice. Tetap gunakan Pipeline walau Decision Tree tidak butuh scaling. Tujuannya agar preprocessing train/valid/test konsisten, terutama untuk encoding fitur kategorikal.

8 Evaluasi untuk Binary Classification

Minimal evaluasi yang dipakai:

- **Confusion matrix:** melihat pola FP/FN.
- **Precision:** penting saat false positive mahal.
- **Recall:** penting saat false negative mahal.
- **F1-score:** kompromi precision-recall.
- **ROC-AUC:** kualitas ranking probabilitas lintas threshold.

Metrik	Kapan diprioritaskan
Precision	Saat ingin menekan false alarm.
Recall	Saat miss positive lebih berbahaya.
F1	Saat butuh keseimbangan precision-recall.
ROC-AUC	Saat submission kompetisi berbasis probabilitas/ranking.

Table 4: Pemilihan metrik bergantung konteks keputusan, bukan sekadar angka tertinggi tunggal.

9 Interpretasi Model Tree

Dua output interpretasi yang wajib ditunjukkan:

1. **Feature importance** untuk melihat fitur yang paling banyak menurunkan impurity.
2. **Decision rules** (jalur root ke leaf) untuk menjelaskan logika prediksi.

Catatan penting: impurity-based feature importance bisa bias ke fitur dengan banyak kategori setelah one-hot. Karena itu, tetap validasi dengan performa out-of-sample.

Referensi

- Han, J., Kamber, M., & Pei, J. *Data Mining: Concepts and Techniques*, 4th ed.
- scikit-learn User Guide – Decision Trees: <https://scikit-learn.org/stable/modules/tree.html>
- scikit-learn API – DecisionTreeClassifier: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>